

SituationReport

testdata_generator

Benutzerhandbuch

Synthetische Jira-Issue-Daten für SituationReport erzeugen

situation-report -- github.com/Jaegerfeld/situation-report

Fuer Agile Coaches und PI Manager -- Version 0.99

1 Was ist der Testdata Generator?

Der **Testdata Generator** erzeugt synthetische Jira-Issue-Daten im Jira-REST-API-Format. Die generierten Dateien sind direkt mit dem Modul **transform_data** verarbeitbar — ohne eine echte Jira-Instanz.

Der Generator simuliert realistische Workflow-Historien: Issues durchlaufen die definierten Stages, verbleiben unterschiedlich lange in jeder Stage, werden gelegentlich zurückgestuft und schließen mit konfigurierbarer Rate ab.

Typische Einsatzgebiete:

- **Neue Installation testen:** Prüfen, ob transform_data und build_reports korrekt arbeiten
- **Demos und Präsentationen:** Realistische Daten ohne Datenschutzbedenken
- **Schulungen:** Lernumgebung mit bekannten, kontrollierten Daten
- **Entwicklung:** Reproduzierbare Testdaten über den Seed-Parameter

Der Testdata Generator ergänzt das Modul **Get Data** (noch in Entwicklung), das echte Daten direkt aus Jira lädt. Für den manuellen Export echter Jira-Daten steht das **Benutzerhandbuch Get Data** zur Verfügung.

2 Voraussetzungen und Start

2.1 Portables Paket (empfohlen)

Das portable Paket enthält Python bereits eingebettet — keine separate Installation erforderlich.

Betriebssystem	Startdatei
Windows	TestdataGenerator.bat (Doppelklick)
macOS	TestdataGenerator.command (Rechtsklick → Öffnen)
Linux	./TestdataGenerator.sh

Hinweis für Windows: Falls beim Start eine Fehlermeldung erscheint, die Python nicht findet — die ZIP-Datei war möglicherweise durch Windows blockiert. Lösung: Rechtsklick auf die ZIP → Eigenschaften → Sicherheit → Zulassen → OK → erneut entpacken.

2.2 Aus dem Quellcode (Python erforderlich)

```
python -m testdata_generator
```

2.3 Die Benutzeroberfläche

Nach dem Start öffnet sich das Hauptfenster mit Eingabefeldern für alle Parameter. Pflichtfeld ist nur die Workflow-Datei — alle anderen Felder haben sinnvolle Standardwerte.

testdata_generator 0.9.2

Optionen / Options

Workflow-Datei

Ausgabedatei (JSON)

Projekt-Key

Anzahl Issues

Von (YYYY-MM-DD)

Bis (YYYY-MM-DD)

Issue-Typen (Typ:Gewicht ...)

Completion-Rate (0-1)

To-Do-Rate (0-1)

Backflow-Prob. (0-1)

Seed (leer = zufällig)

Generieren

Log

Abb. 1: Benutzeroberfläche des Testdata Generators

3 Die Workflow-Datei

Die Workflow-Datei definiert die Stages Ihres Jira-Boards und ist das einzige Pflichtfeld des Generators. Sie hat dasselbe Format wie in `transform_data` — eine Datei genügt für beide Module.

3.1 Format

Jede Zeile beschreibt eine Stage. Aliases (alternative Status-Namen im Jira-Export) werden durch Doppelpunkte abgetrennt. Zwei Sonderzeilen markieren Anfang und Ende der aktiven Bearbeitung:

```
KanonischerName:Alias1:Alias2
<First>ErsteAktiveStage
<Closed>AbgeschlosseneStage
```

Zeile	Bedeutung
KanonischerName:Alias1:Alias2	Stage mit einem oder mehreren alternativen Jira-Status-Namen
StageName (ohne :)	Stage ohne Aliases
<First>StageName	Erste aktive Stage — wo Bearbeitung beginnt (Cycle-Time-Start)
<Closed>StageName	Abgeschlossene Stage — markiert ein Issue als fertig

3.2 ART_A Beispiel-Workflow

Das mitgelieferte Beispiel `workflow_ART_A.txt` bildet einen typischen SAFe-ART-Workflow ab:

```
Funnel:New:Open:To Do
Analysis:In Analysis:Estimated
Program Backlog:Ready for Dev:Backlog
Implementation:In Implementation:In Review:In Progress
Blocker
Validating on Staging:QA
Deploying to Production:Ready for Development
Releasing:Completed
Done:Canceled
<First>Analysis
<Closed>Releasing
```

Anmerkungen: 'Funnel' enthält vier Aliases — alle vier Jira-Status-Namen werden auf 'Funnel' normiert. Analysis ist die erste aktive Stage (Cycle-Time beginnt hier). Releasing gilt als abgeschlossen. Done (nach Releasing) nimmt auch Canceled-Issues auf.

4 Parameter-Referenz

Alle Parameter sind sowohl in der GUI als auch auf der Kommandozeile verfügbar.

Parameter	Standard	Beschreibung
--workflow FILE	(Pflicht)	Workflow-Definitionsdatei (.txt)
--output FILE.json	<project>_generated.json	Pfad der Ausgabedatei
--project KEY	TEST	Jira-Projekt-Key der generierten Issues
--issues N	100	Anzahl zu generierender Issues
--from-date YYYY-MM-DD	2025-01-01	Frühestes Erstellungsdatum
--to-date YYYY-MM-DD	2025-12-31	Spätestes Übergangsdatum
--issue-types TYPE:W ...	Feature:0.6 Bug:0.3 Enhancement:0.1	Issue-Typen mit Gewichtung (Summe muss nicht 1 ergeben)
--completion-rate FLOAT	0.7	Anteil Issues, die die Closed-Stage erreichen (0–1)
--todo-rate FLOAT	0.15	Anteil offener Issues, die in frühen Stages verbleiben (0–1)
--backflow-prob FLOAT	0.1	Wahrscheinlichkeit eines Rückschritts bei jedem Übergang (0–1)
--seed INT	(zufällig)	Seed für reproduzierbare Ausgabe (optional)
--mean-cycle-days FLOAT	(keiner)	Mittlere Cycle-Time in Tagen (lognormal); aktiviert CT-Steuerung
--std-cycle-days FLOAT	(30 % des Mittels)	Standardabweichung der Cycle-Time in Tagen
--pattern MUSTER	none	Flow-Antipattern: none / triangle / flat_triangle / cluster / batch
--pi-duration-weeks INT	12	PI-Zyklus-Länge in Wochen (für cluster/batch)

Tipp: Reproduzierbare Ergebnisse

Mit einem festen Seed (z. B. --seed 42) erzeugt der Generator bei gleichen Parametern immer dieselbe JSON-Datei — ideal für Demos, bei denen alle Teilnehmer dieselben Ergebnisse sehen sollen.

4b Flow-Antipattern-Muster

Ab Version 0.99 kann der Testdata Generator typische Flow-Antipatterns aus Vacanti/Singh simulieren. Dazu wird **--mean-cycle-days** kombiniert mit **--pattern**.

Muster	Scatterplot-Erscheinung	Erklärung
none	Gleichmäßige Punktwolke	Zufällige Cycle-Time ohne Trend (Standard)
triangle	Dreieck mit rechtem Winkel unten links	CT steigt linear über die Zeit: $\text{mult} = 1 + 3 \times t$
flat_triangle	Dreieck, aber Anstieg flacht am Ende ab	Anstieg wird durch $\tanh(2.5 \times t)$ gedämpft
cluster	Häufungen senkrechter Punkte am Ende	CT ist stabil, aber Abschlüsse in letzten 2 Wochen eines PI
batch	Senkrechte Säulen mit variabler CT und PI-Ende	CT um PI-Ende bis $3 \times$ Mittelwert + PI-Clustering

Beispiele

```
# Triangle-Muster
python -m testdata_generator \
  --workflow workflow.txt --issues 300 \
  --pattern triangle --mean-cycle-days 20 --std-cycle-days 6 \
  --output triangle.json

# Cluster of Dots (12-Wochen-PI)
python -m testdata_generator \
  --workflow workflow.txt --issues 300 \
  --pattern cluster --mean-cycle-days 30 \
  --pi-duration-weeks 12 --output cluster.json

# Batch Transfers
python -m testdata_generator \
  --workflow workflow.txt --issues 300 \
  --pattern batch --mean-cycle-days 30 \
  --pi-duration-weeks 12 --output batch.json
```

Hinweis: --pattern erfordert --mean-cycle-days. Ohne --mean-cycle-days wird pattern ignoriert und das bisherige min/max-Dwell-Verhalten verwendet.

5 ART_A – Vollständiges Beispiel

Dieses Beispiel zeigt den vollständigen Ablauf vom Generator bis zum fertigen Report am fiktiven Projekt ART_A.

Schritt 1: Testdaten generieren

```
python -m testdata_generator \  
    --workflow testdata_generator/workflow_ART_A.txt \  
    --project ART_A \  
    --issues 200 \  
    --from-date 2025-01-01 \  
    --to-date 2025-12-31 \  
    --issue-types Feature:0.6 Bug:0.2 Enabler:0.2 \  
    --completion-rate 0.75 \  
    --backflow-prob 0.08 \  
    --seed 42 \  
    --output ART_A_generated.json
```

Ergebnis: ART_A_generated.json mit 200 Issues, ca. 150 abgeschlossen und 50 in verschiedenen Stages offen. Dank Seed 42 ist die Ausgabe jederzeit reproduzierbar.

Schritt 2: Mit transform_data verarbeiten

```
python -m transform_data ART_A_generated.json \  
    --workflow testdata_generator/workflow_ART_A.txt
```

Erzeugt drei Excel-Dateien: ART_A_generated_IssueTimes.xlsx, ART_A_generated_CFD.xlsx und ART_A_generated_Transitions.xlsx.

Schritt 3: Report mit build_reports erstellen

```
python -m build_reports
```

In build_reports die drei Excel-Dateien aus Schritt 2 laden — das Programm erstellt daraus Flow-Diagramme, Cycle-Time-Analysen und Metriken.

Was zu erwarten ist (Seed 42, 200 Issues):

- Issue-Keys: ART_A-0001 bis ART_A-0200
- Issue-Typen: ~120 Feature, ~40 Bug, ~40 Enabler
- Ca. 150 Issues im Status 'Releasing' oder 'Done'
- Erstellungsdaten verteilt über das Jahr 2025
- Gelegentliche Rückschritte (backflow-prob 0.08 = ca. 8 %)

6 Häufige Fragen (FAQ)

Unterschied zwischen Testdaten und echten Daten?

Für Demos, Schulungen und Installationstests sind Testdaten die bessere Wahl — keine Datenschutzbedenken, reproduzierbar und kontrolliert. Für echte Flow-Analysen und Entscheidungen spiegeln echte Daten den tatsächlichen Workflow-Zustand wider.

Für den manuellen Export echter Jira-Daten (API-Token, curl, Paginierung): Siehe Benutzerhandbuch Get Data.

7 Glossar

Begriff	Erklärung
ART	Agile Release Train — ein Team aus mehreren Scrum-Teams in SAFe
API	Application Programming Interface — Programmierschnittstelle
API-Token	Sicherheitsschlüssel für den Zugang zur Jira-REST-API; ersetzt das Passwort bei curl-Abfragen
Changelog	Jira-Protokoll aller Statusänderungen eines Issues (expand=changelog)
CFD	Cumulative Flow Diagram — zeigt den Issuebestand je Stage kumuliert über die Zeit
Closed Stage	Die Stage, die ein Issue als abgeschlossen markiert (workflow.txt: <First> bis zur Closed-Stage)
Cycle Time	Zeit von der ersten aktiven Stage (<First>) bis zur Closed-Stage
First Stage	Erste aktive Stage, ab der die Cycle Time zählt (workflow.txt: <First>)
Helper	SituationReport-Modul zum Zusammenführen mehrerer Jira-JSON-Dateien
Issue	Arbeitselement in Jira (Feature, Bug, Enabler, Story, ...)
JQL	Jira Query Language — Abfragesprache für Jira-Suchen, z. B. project=ART_A
JSON	JavaScript Object Notation — Format der Jira-API-Exporte
Seed	Startwert für den Zufallsgenerator — gleicher Seed ergibt immer dieselbe Ausgabe
Stage	Ein Schritt im Workflow (entspricht einer Jira-Spalte oder einem Status-Namen)
Workflow	Geordnete Abfolge von Stages, die ein Issue durchläuft